

大模型学习路线

Agent 开发 模型微调 优化部署

学习者	海南大学 2025 级软件工程 NIIT 本科生
现有基础	Python requests FastAPI MySQL
核心目标	先完成 Agent 应用，再深入微调与部署
整理日期	2026 年 9 月 17 日
推进方式	理论与实践并行 按阶段成果验收

1 学习目标与整体安排

本路线以 Agent 开发为主线，衔接已有 Python、requests、FastAPI 和 MySQL 基础。先学会调用模型、检索知识和执行工具，再补齐模型训练原理，完成微调和自行部署，最后按兴趣深入对齐、强化学习、多模态或推理工程。

学习不按固定周数推进。每一阶段都通过项目、解释和测试验收；阶段 1—2 期间同步补阶段 3 的基础，进入 SFT 前再把训练原理补到能解释、能修改的程度。

阶段	实践主线	同期理论	完成标志
1 模型调用	对话接口与结构化输出	token 上下文 生成参数	独立完成可测试的对话后端
2 Agent 与 RAG	工具调用与知识检索	向量表示 检索与生成	基于资料 and 工具结果完成任务
3 训练基础	PyTorch 与小模型实验	反向传播 Attention Transformer	能解释并修改训练过程
4 SFT 微调	数据处理 LoRA QLoRA	损失 显存 过拟合	完成可复现的对照实验
5 优化部署	模型服务与性能测试	KV cache 批处理	接回 Agent 并给出性能报告
6 专项深入	DPO RL 多模态等选题	偏好学习 分布式训练	围绕具体问题深入实验

需要先分清的概念

开发 Agent、开发对话系统、微调模型与自行部署模型可以分别完成。前期通过现成模型 API 就能开发 Agent，后续再替换模型或接入微调版本。

RNN、CNN、Seq2Seq 无须全部精通后才能学习 Transformer。先理解其问题与局限，把主要精力放在 PyTorch、注意力机制和 Transformer。

Decoder-only 的典型特点是自回归预测后续 token，不代表天然低延迟。模型规模、输出长度、硬件与推理实现都会影响实际速度。[1]

评估从第一个项目开始；是否增加 RAG、微调或多 Agent，由失败案例和评测结果决定。

2 从模型调用到 Agent 应用

阶段 1 把后端基础接到大模型上

重点学习模型 API 与 SDK、消息角色、多轮上下文、token 和上下文窗口、输出长度与温度、结构化输出、JSON Schema 和 Pydantic 验证。工程侧补流式响应、超时重试、日志、环境变量、模块拆分、Git 与接口测试，以及 async/await 的基本用法。

模型认知先抓实用区别：基础模型与指令模型、生成模型与 Embedding 模型、云端 API 与开放权重模型。用实际任务比较中文能力、工具调用、费用、速度和质量。

实践项目：用 FastAPI 建立支持多轮对话和结构化信息提取的接口。先保留现有学生信息项目，新增一个可单独运行的模型调用模块。

验收标准：正常请求能完成；超时、参数错误和输出格式错误能被处理；记录请求耗时；使用固定样例比较两种模型或两种提示词。

阶段 2 先完成单 Agent 再加入 RAG

先用普通 Python 和模型 SDK 实现循环：用户提出任务 → 模型选择工具 → 程序验证并执行 → 返回工具结果 → 模型继续或结束。学习工具参数、上下文状态、执行步数限制、失败处理和执行记录；理解 ReAct 中推理与行动交替推进的思想。[2]

工具与编排：接入自己编写的查询接口；单 Agent 做稳定后，再学 LangGraph 的状态与流程编排、MCP 工具接入。多 Agent 在任务确实需要分工时再增加。

RAG 学习清单：文档读取、清洗、切分与来源保存；Embedding、相似度、向量检索、关键词检索；Top-k、重排序和引用。要区分“没有检索到证据”与“有证据却回答错误”。

实践项目：课程资料与学习记录助手。它可以检索课程笔记、查询学习记录、生成带来源的回答，再通过 FastAPI 保存学习记录。

验收标准：建立约 30—50 个固定任务，覆盖正常问题、资料不足、参数缺失和工具失败。检查引用是否支持结论、工具与参数是否正确、任务是否真正完成。

3 深度学习与 Transformer 基础

学习顺序

向量与矩阵、导数与链式法则、概率基础 → 线性模型、损失函数、梯度下降 → 神经网络、反向传播、过拟合 → PyTorch → Embedding、Attention 与 Transformer。数学知识结合代码实验学习，遇到公式时核对每个量的含义和维度。

PyTorch 必须亲手完成的过程

Dataset 与 DataLoader → forward → loss → backward → optimizer.step。掌握 Tensor 形状、设备与数据类型、自动求导、训练和评估模式、模型保存与加载。[3]

先完成小型分类或回归模型，画训练与验证曲线，尝试调整学习率和模型容量。能说明模型参数、梯度与优化器状态分别是什么。

Transformer 的核心清单

分词与 token ID、词表、Embedding；Q、K、V；缩放点积注意力、多头注意力与因果掩码；位置编码、残差连接、归一化和前馈网络。

理解 Encoder、Decoder 与 Encoder-Decoder 的区别；下一 token 预测的训练目标；训练时的并行计算与自回归生成的区别；输入上下文如何变成输出概率。[1]

RNN、LSTM 与 Seq2Seq 以概念和小实验为主，理解序列建模和信息瓶颈。CNN 学基础结构，后续需要多模态时再深入。

实践项目与验收

先训练一个小型分类模型，再实现简化的注意力层或小型语言模型。小模型的目标是理解训练流程，生成质量可以有限。

验收时能回答：张量经过每层后的维度是什么；梯度如何更新参数；为什么训练损失下降而验证效果变差；因果掩码遮住了什么；模型训练和推理分别保存或计算哪些内容。

4 完成一次可复现的 SFT 微调

前提是能够解释基本训练过程，并已建立原模型在目标任务上的评测基线。选择一个明确弱点开展实验，例如结构化信息提取或特定工具参数生成。

环节	必须掌握的内容
数据准备	来源与使用权限、采集、清洗、去重、质量检查、结构化对话格式
数据划分	训练 验证 测试集；按来源或相近样本分组，防止跨集合泄漏
数据编码	tokenizer、chat template、截断、padding、损失计算范围
训练设置	学习率调度、AdamW、batch size、混合精度、梯度累积
参数高效微调	先理解 LoRA，再实践 QLoRA；Prompt Tuning 先了解原理
实验管理	数据和模型版本、配置、随机种子、日志、检查点

理解 SFT 与参数高效微调的关系

SFT 描述监督微调的训练任务，LoRA 描述只更新部分新增参数的方式，两者可以结合。QLoRA 结合量化基础模型和 LoRA 适配器训练；Prompt Tuning 是训练软提示参数，与编写普通文本提示词不同。[4][5]

完整实验流程

确定任务与指标 → 准备并审核样本 → 固定数据划分 → 评估原模型 → 配置并训练 → 保存模型或适配器 → 在未参与训练的测试集评估 → 接回应用并记录失败案例。

模型大小、序列长度和训练方案根据实际显存决定。先用小规模实验确认数据格式、损失与加载流程，再扩大训练量。

实践项目与验收

比较“原模型加提示词”和“微调模型”，记录提升、退步、资源消耗和泛化表现。训练命令成功结束、loss 下降，都不能单独证明微调有效。

交付一份实验记录：任务定义、数据说明、模型与配置、训练曲线、测试结果、典型失败案例、复现命令。测试集固定后不再反复用它调参。

5 自行部署与性能优化

基础模型的本地推理可以提前尝试；本阶段集中学习服务化和系统性能。目标是把自己部署的模型接入同一个 Agent 项目，并能解释优化的收益与代价。

模型服务与工程基础

核对模型、tokenizer、chat template 与适配器的版本对应；理解模型推理服务与 FastAPI 业务服务的分工；学习 Linux、容器和部署环境的基础使用。

加入并发限制、超时、健康检查、故障恢复与版本回滚。把日志用于定位模型调用、工具执行、检索和业务接口的故障。

推理机制与性能优化

学习 prefill 与 decode、KV cache 的作用与显存成本、连续批处理、上下文长度对资源占用的影响。量化要同时检查效果、显存和速度，不能只看模型文件大小。

可用 vLLM 学习服务与批处理，其官方文档覆盖 KV cache 管理、连续批处理和量化等能力。[6]

指标	测量时要说明的条件
首 token 延迟	相同输入长度与并发条件下，从请求到首个输出的时间
生成速度	后续 token 的生成速度，并说明输出长度与统计口径
吞吐与并发	单位时间完成的请求或 token 数，同时观察尾部延迟
资源与质量	显存、稳定性和原有任务测试集上的效果

实践项目与验收

将自行部署的模型接回 Agent；固定硬件、输入输出长度和并发条件，比较量化前后或不同服务配置的表现。一次实验尽量只改变一个关键因素。

交付部署说明、启动方式、配置版本、性能报告和故障恢复步骤。验收时应能解释：优化了什么、牺牲了什么、在哪些请求下有效。

6 对齐与专项深入

本阶段按兴趣和实际项目需要选择一个方向深入。分布式训练、大模型预训练和复杂多 Agent 都不是完成前五阶段的必需条件。

分支	建议学习顺序
偏好对齐	理解 RLHF 流程 → 构造偏好数据 → 实践 DPO → 深入强化学习训练
Agent 研究	单 Agent 评测 → 长任务与状态恢复 → 多 Agent 协作 → 训练改善决策
多模态	图文表示 → CLIP 匹配与检索 → 视觉语言模型 → 多模态 Agent
模型压缩	量化实践 → 蒸馏 → 按具体模型研究剪枝
分布式训练	数据并行 → ZeRO 与 FSDP → 张量并行与流水线并行
持续迭代	收集反馈 → 审核样本 → 离线评估训练 → 发布回滚 → 再考虑在线学习

偏好对齐与强化学习

DPO 使用同一问题的偏好回答对直接优化模型，不要求先搭建独立奖励模型和 PPO 训练流程，适合放在第一次 SFT 实验之后。[7]

强化学习分支依次补环境、状态、动作、奖励、策略与价值函数，先在小型环境比较学习策略和随机策略，再深入 DQN、PPO 或多智能体。进入大模型强化学习前，应能分析训练曲线、奖励设计与实验波动。

多模态与计算资源

CLIP 用于理解图文表示对齐、相似度和检索；图像对话还要进一步学习视觉语言模型。[8]

分布式训练先理解不同方案切分的是数据、参数、梯度还是优化器状态；需要多卡实验时再深入配置。在线学习放在反馈质量、离线评估、版本管理和回滚机制已经可靠之后。

7 评估方法与当前行动

评估贯穿整条路线

对象	主要检查内容
对话与结构化输出	答案正确性、格式遵循、要求遗漏、内容理解
RAG	证据召回、引用支持程度、无证据时的处理、幻觉
Agent	工具选择与参数、执行结果、任务成功率、步数和费用
微调模型	未见测试集表现、泛化、原有能力退步与失败类型
安全与隐私	越权调用、敏感数据泄漏、提示注入和不当内容响应

初期采用人工检查和简单断言，之后再加入自动评分；模型自动评分需要抽样人工核对。固定测试集、记录配置并保存每次结果，才能比较改动是否有效。

当前最优先完成的四件事

- 第一步：整理现有 FastAPI 项目，分开配置、数据模型、业务逻辑与数据库访问；为正常输入、错误输入和不存在的数据增加可重复验证。
- 第二步：新增一个模型调用模块，完成一次请求、结构化解析、异常处理与耗时记录，再用 FastAPI 对外提供接口。
- 第三步：把自己写的一个查询接口封装成工具，观察模型如何选工具、程序如何执行、结果如何回传，并设置终止条件。
- 第四步：理论侧学习 PyTorch Tensor、自动求导与训练循环，完成一个小模型实验并画出训练和验证曲线。

统一项目与阶段记录

围绕“课程资料与学习记录助手”持续迭代：对话接口 → 工具调用 → RAG → 微调实验 → 自行部署 → 专项优化。保留每个版本及其评测结果，避免为每个新技术重新开始一个无关项目。

每完成一个阶段记录：本次成果、能独立解释的知识、仍需帮助的部分、测试证据、典型失败、下一步。以理解和结果作为进入下一阶段的依据。

参考资料

以下为路线中引用的官方文档与原始论文。优先按当前阶段阅读对应部分，实践时固定依赖版本并核对当时的接口。

1 Transformer 架构

用于理解 Encoder、Decoder 与 Encoder-Decoder。

<https://huggingface.co/learn/llm-course/chapter1/6>

2 ReAct 原始论文

用于理解推理与行动交替推进的智能体思路。

<https://arxiv.org/abs/2210.03629>

3 PyTorch 官方基础教程

用于张量、数据加载、自动求导、优化与模型保存。

<https://docs.pytorch.org/tutorials/beginner/basics/intro.html>

4 TRL SFT Trainer

用于指令微调、对话数据格式、损失与适配器训练。

https://huggingface.co/docs/trl/sft_trainer

5 PEFT 量化指南

用于理解量化基础模型与 LoRA 适配器训练。

https://huggingface.co/docs/peft/developer_guides/quantization

6 vLLM 官方文档

用于推理服务、缓存、批处理与性能优化。

<https://docs.vllm.ai/en/latest/>

7 TRL DPO Trainer

用于偏好数据与直接偏好优化。

https://huggingface.co/docs/trl/dpo_trainer

8 CLIP 原始论文

用于图文表示对齐、匹配和检索。

<https://arxiv.org/abs/2103.00020>